Ex.No:6	Orchestrating Processes: The Symphony of System Calls and Control.
Date :	URK23CS1302

Aim:

To understand process creation, parent-child relationships, and process management using system calls such as fork(), getpid(), and wait() in Unix/Linux.

Theme: Online learning tracking system.

Description:

The **Online Learning Tracking System** orchestrates various processes using system calls and control mechanisms to monitor students' learning activities efficiently. The system acts as a symphony where each system call represents a musical note, coordinating tasks like student login, session tracking, and activity logging. It uses process creation to start learning sessions, inter-process communication (IPC) to exchange data between modules, and file system calls to store attendance and performance reports. This system ensures seamless tracking of students' progress while maintaining secure and synchronized data flow throughout the learning journey.

Exercise:

Question 1: Write a program to create a child process that performs a specific task related to your theme, like issuing a book or updating stock.

```
 GNU nano 8.2                    exp6_1.sh
#!/bin/bash

# Function to simulate issuing a book
issue_book() {
    echo "Child Process: Checking book availability..."
    sleep 2  # Simulating processing time
    echo "Child Process: Book issued successfully!"
}

# Creating a child process
issue_book &  # Run issue_book function in the background
child_pid=$!  # Store the child process ID

# Parent process waits for child to complete
echo "Parent Process: Waiting for the child process to complete..."
wait $child_pid
echo "Parent Process: Book issue process completed. Exiting..."
```

```
┌──(kali㉿kali)-[~/exp6]
└─$ nano exp6_1.sh

┌──(kali㉿kali)-[~/exp6]
└─$ chmod +x exp6_1.sh

┌──(kali㉿kali)-[~/exp6]
└─$ ./exp6_1.sh
Parent Process: Waiting for the child process to complete...
Child Process: Checking book availability...
Child Process: Book issued successfully!
Parent Process: Book issue process completed. Exiting...
```

Question 2:Createamulti-processprogramwhereeachprocesshandlesa different operationinyour theme(e.g.,cataloging,borrowing,orreturning).

kali@kali: ~ × kali@kali: ~/exp5 × kali@kali: ~/exp5 × kali@kali: ~/exp6 ×

GNU nano 8.2 exp6_2.sh

```
#!/bin/bash

# Function for cataloging books
catalog_books() {
    echo "Cataloging books ..."
    sleep 2
    echo "Books cataloged."
}

# Function for borrowing books
borrow_books() {
    echo "Processing book borrowing ..."
    sleep 3
    echo "Book borrowed successfully."
}

# Function for returning books
return_books() {
    echo "Processing book return ..."
    sleep 2
    echo "Book returned successfully."
```

```
┌──(kali㉿kali)-[~/exp6]
└─$ nano exp6_2.sh

┌──(kali㉿kali)-[~/exp6]
└─$ chmod +x exp6_2.sh

┌──(kali㉿kali)-[~/exp6]
└─$ ./exp6_2.sh
Cataloging books ...
Processing book borrowing ...
Processing book return ...
Book returned successfully.
Books cataloged.
Book borrowed successfully.
All operations completed.
```

Question 3:Use fork() to create child processes and print the PID and PPID for each. Log these details in a theme-related file.

GNU nano 8.2 exp6_3.sh

```
#!/bin/bash

log_file="process_log.txt"

# Clear previous log file
> "$log_file"

echo "Parent Process: PID = $$" | tee -a "$log_file"

# Creating a child process
child_pid=$(bash -c 'echo $$')  # Simulates a fork by running a subshell

echo "Child Process: PID = $child_pid, PPID = $$" | tee -a "$log_file"
```

```
┌──(kali㉿kali)-[~/exp6]
└─$ nano exp6_3.sh

┌──(kali㉿kali)-[~/exp6]
└─$ chmod +x exp6_3.sh

┌──(kali㉿kali)-[~/exp6]
└─$ ./exp6_3.sh
Parent Process: PID = 170116
Child Process: PID = 170119, PPID = 170116
```

Question 4:Implement a program to track and display the status of theme-related processes (e.g., running, stopped, or terminated).

```
  GNU nano 8.2                          exp6_4.sh
#!/bin/bash

log_file="process_status.log"

# Clear previous log file
> "$log_file"

echo "Tracking theme-related processes..." | tee -a "$log_file"

# List all processes related to 'theme' (replace 'theme' with your actual proce
ps aux | grep "[t]heme" | tee -a "$log_file"

echo -e "\nProcess Status Summary:" | tee -a "$log_file"
ps -eo pid,comm,state | grep "[t]heme" | tee -a "$log_file"

echo -e "\nProcess tracking completed. Status saved in $log_file"
```

```
┌──(kali㉿kali)-[~/exp6]
└─$ nano exp6_4.sh

┌──(kali㉿kali)-[~/exp6]
└─$ chmod +x exp6_4.sh

┌──(kali㉿kali)-[~/exp6]
└─$ ./exp6_4.sh
Tracking theme-related processes...

Process Status Summary:

Process tracking completed. Status saved in process_status.log
```

Question 5:Simulate parallel executionof theme-related tasks usingmultiple processes.

```
  GNU nano 8.2                          exp6_7.sh
    sleep 3
    echo "Cataloging process completed."
}

# Function for borrowing
borrowing() {
    echo "Borrowing process (PID $$) started..."
    sleep 4
    echo "Borrowing process completed."
}

# Function for returning
returning() {
    echo "Returning process (PID $$) started..."
    sleep 5
    echo "Returning process completed."
}

# Run all tasks in parallel using background execution
cataloging &
borrowing &
```

```
┌──(kali㉿kali)-[~/exp6]
└─$ nano exp6_7.sh

┌──(kali㉿kali)-[~/exp6]
└─$ chmod +x exp6_7.sh

┌──(kali㉿kali)-[~/exp6]
└─$ ./exp6_7.sh
Starting parallel execution of theme-related tasks...
Cataloging process (PID 174093) started...
Borrowing process (PID 174093) started...
Returning process (PID 174093) started...
Cataloging process completed.
Borrowing process completed.
Returning process completed.
All theme-related tasks completed.
```

Question 6: Perform sorting of files based on their size or name and save the sorted list in a new file.

```
  GNU nano 8.2                          exp6_8.sh
#!/bin/bash

echo "Calculating total execution time for theme-related processes..."

# Start time
start_time=$(date +%s)

# Simulating theme-related processes
cataloging() {
    echo "Cataloging process started..."
    sleep 3
    echo "Cataloging process completed."
}

borrowing() {
    echo "Borrowing process started..."
    sleep 4
    echo "Borrowing process completed."
}

returning() {
                                [ Read 41 lines ]
```

```
┌──(kali㉿kali)-[~/exp6]
└─$ nano exp6_8.sh

┌──(kali㉿kali)-[~/exp6]
└─$ chmod +x exp6_8.sh

┌──(kali㉿kali)-[~/exp6]
└─$ ./exp6_8.sh
Calculating total execution time for theme-related processes...
Cataloging process started...
Borrowing process started...
Returning process started...
Cataloging process completed.
Borrowing process completed.
Returning process completed.
Total execution time: 5 seconds.
```

Question 7: Simulate parallel execution of theme-related tasks using multiple processes.

```
  GNU nano 6.0                                  e67.sh *
#!/bin/bash

task1() {
    echo "Downloading course materials..."
    sleep 2
    echo "Download complete!"
}

task2() {
    echo "Verifying user progress..."
    sleep 3
    echo "Verification done!"
}

task1 & task2 &
wait
```

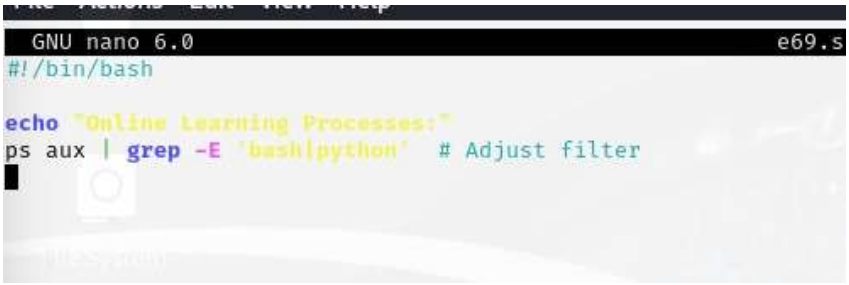
```
kali% ./e67.sh
Downloading course materials...
Verifying user progress...
Download complete!
Verification done!
kali%
```

Question 8: Write a program to calculate the total execution time of theme-related processes.




```
kali% ./e68.sh
Processing...
Done!
Total Time: 3 seconds
kali%
```

Question 9: Create a script to generate a report of all running processes and identify those associated with your theme.




```
kali% ./e69.sh
Online Learning Processes:
kali        1057  0.0  1.2 372368 50704 ?        Sl   Mar23   0:00 /usr/bin/python3 ,
kali       17398  0.0  0.0   7008  3428 pts/0    S+   03:16   0:00 /bin/bash ./e69.sl
kali       17400  0.0  0.0   6320  2228 pts/0    S+   03:16   0:00 grep -E bash|pyth
kali%
```

Question 10: Implement inter-process communication between parent and child processes to pass data relevant to your theme.



Output:



Result:

The experiment to perform process creation, parent-child relationships, and process management using system calls has been executed successfully and output is verified